

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)

Assessment Tool Weightage: 35%

Faculty Name: Dr.G.Ayyappan

Student Reg.No: 192424061

Name:K.Sandhya

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5

8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

*I _K.Sandhya 192424061_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Complete Query Processing Pipeline

Aim:

To write a pseudocode for processing an SQL query from input to final result.

Algorithm:

1. Start
2. Read the SQL query.
3. Parse the SQL query and check syntax.
4. Create the parse tree.
5. Rewrite and simplify the query.
6. Generate different execution plans.
7. Estimate the cost of each plan.
8. Select the plan with minimum cost.
9. Execute the selected query plan.
10. Display the query result.
11. Stop.

Code :

ALGORITHM Query_Processing(SQL_Query)

INPUT: SQL query

OUTPUT: Query result

BEGIN

1. Receive SQL_Query from user
2. PARSING
 - Perform lexical analysis
 - Check SQL syntax
 - Check table and column names
 - Create Parse Tree

3. QUERY REWRITING

- Simplify the query
- Apply selection and projection rules
- Create optimized logical query tree

4. QUERY OPTIMIZATION

- Generate different execution plans
- Estimate cost of each plan
- Select the plan with minimum cost

5. CODE GENERATION

- Convert selected plan into executable operations

6. QUERY EXECUTION

- Access required tables
- Perform joins, selections and projections
- Generate result

7. RETURN RESULT

- Send the result to the user

END

Result:

Thus, the SQL query was successfully processed through parsing, optimization, execution, and result generation.

2. Cost-Based Query Optimization

Aim:

To design a pseudocode algorithm for selecting the query execution plan with minimum I/O cost.

Algorithm:

1. Start

2. Read the relations and their statistics.
3. Generate possible join orderings.
4. Calculate the I/O cost for each join ordering.
5. Compare all calculated costs.
6. Select the join ordering with minimum cost.
7. Return the best execution plan.
8. Stop.

Code :

ALGORITHM Cost_Based_Optimization

INPUT: Relations R1, R2, ..., Rn

OUTPUT: Best join ordering

BEGIN

 Generate all possible join orderings

 best_cost $\leftarrow$ INFINITY

 best_plan $\leftarrow$ NULL

 FOR each join_order in join_orders DO

 cost $\leftarrow$ 0

 FOR each join in join_order DO

 Estimate number of pages

 Estimate number of tuples

 Calculate I/O cost of the join

 cost $\leftarrow$ cost + join_cost

 END FOR

 IF cost < best_cost THEN

 best_cost $\leftarrow$ cost

 best_plan $\leftarrow$ join_order

 END IF

 END FOR

 RETURN best_plan, best_cost

END

Result:

Thus, the optimal join ordering with the minimum estimated I/O cost was successfully selected.

3. Nested Loop Join**Aim:**

To write pseudocode for the Nested Loop Join algorithm and analyze its time complexity.

Algorithm:

1. Start
2. Read relations R and S.
3. Select each tuple r from R.
4. For every r, select each tuple s from S.
5. Check the join condition.
6. If the condition is true, add (r,s) to the result.
7. Repeat until all tuples are processed.
8. Display the result.
9. Stop.

Time Complexity:

For T(R) and T(S) tuples:

$$O(T(R) \times T(S))$$

Code :

ALGORITHM Nested_Loop_Join(R, S)

INPUT: Relations R and S

OUTPUT: Joined relation

BEGIN

FOR each tuple r in R DO

FOR each tuple s in S DO

IF JoinCondition(r, s) = TRUE THEN

Add (r, s) to Result

END IF

```
        END FOR
    END FOR
    RETURN Result
END
```

Result:

Thus, the Nested Loop Join was successfully performed, and its time complexity was determined as $O(T(R) \times T(S))$.

4. Deadlock Detection Using WFG

Aim:

To detect deadlocks in a transaction system using a Wait-For Graph.

Algorithm:

1. Start
2. Create the Wait-For Graph.
3. Mark all transactions as unvisited.
4. Perform DFS on each transaction.
5. Check whether a cycle exists.
6. If a cycle is found, report deadlock.
7. Otherwise, report no deadlock.
8. Stop.

Code :

ALGORITHM Detect_Deadlock(WFG)

INPUT: Wait-For Graph

OUTPUT: Deadlock or No Deadlock

BEGIN

 Mark all transactions as UNVISITED

 FOR each transaction T DO

 IF T is UNVISITED THEN

 IF DFS(T) finds a cycle THEN

 PRINT "Deadlock Detected"

 RETURN

```
        END IF
    END IF
END FOR
PRINT "No Deadlock"
END
```

DFS procedure:

```
DFS(T)
BEGIN
    Mark T as VISITING
    FOR each transaction U connected from T DO
        IF U is VISITING THEN
            RETURN TRUE
        END IF
        IF U is UNVISITED THEN
            IF DFS(U) = TRUE THEN
                RETURN TRUE
            END IF
        END IF
    END FOR

    Mark T as VISITED
    RETURN FALSE
END
```

Key point:

A cycle in the Wait-For Graph means deadlock.

Result:

Thus, deadlock was successfully detected by checking for a cycle in the Wait-For Graph.

5. Basic Two-Phase Locking

Aim:

To implement the Basic Two-Phase Locking protocol for transaction concurrency control.

Algorithm:

1. Start
2. Begin the transaction.
3. Request a lock for a required data item.
4. If the lock is available, grant the lock.
5. Otherwise, make the transaction wait.
6. Continue acquiring locks during the Growing Phase.
7. Start the Shrinking Phase.
8. Release the required locks.
9. Do not acquire any new locks after releasing a lock.
10. Commit the transaction.
11. Stop.

Code:

ALGORITHM Two_Phase_Locking(Transaction T)

BEGIN

 // Growing Phase

 WHILE T needs a data item DO

 Request lock

 IF lock is available THEN

 Grant lock

 ELSE

 Put T into WAIT state

 END IF

 END WHILE

 // Shrinking Phase

 WHILE T has locked data items DO

 Release lock

```
END WHILE
    COMMIT transaction
END
```

Important rule:

Once T releases its first lock,
T cannot obtain any new lock.

Result:

Thus, the Basic Two-Phase Locking protocol was successfully implemented using growing and shrinking phases.

6. Timestamp-Based Concurrency Control

Aim:

To implement timestamp-based concurrency control using Thomas' Write Rule.

Algorithm:

1. Start
2. Assign a timestamp to each transaction.
3. Read the timestamp of the data item.
4. Check the read/write timestamp conditions.
5. If an old write is obsolete, ignore the write using Thomas' Write Rule.
6. Otherwise, perform the write operation.
7. Update the write timestamp.
8. Continue until all operations are completed.
9. Stop.

Code:

```
ALGORITHM Thomas_Write_Rule(T, X)
INPUT: Transaction T and data item X
BEGIN
IF TS(T) < Read_TS(X) THEN
    ABORT T
```

```

    RETURN
END IF
IF TS(T) < Write_TS(X) THEN
    IGNORE the write
    RETURN
END IF
Perform WRITE(X)
Write_TS(X) ← TS(T)
END

```

For a read:

```

ALGORITHM Read(T, X)
BEGIN
    IF TS(T) < Write_TS(X) THEN
        ABORT T
    ELSE
        READ(X)
        Read_TS(X) ← MAX(Read_TS(X), TS(T))
    END IF
END

```

Thomas Write Rule:

If an old transaction tries to write a value that has already been written by a newer transaction, the old write can be ignored instead of aborting the transaction.

Result:

Thus, timestamp-based concurrency control was successfully implemented using Thomas' Write Rule to handle conflicting operations.

7. ARIES Recovery

Aim:

To implement database recovery using the ARIES technique with Analysis, Redo, and Undo phases.

Algorithm:

1. Start
2. Read the log from the last checkpoint.
3. Perform the Analysis phase.
4. Identify active transactions and dirty pages.
5. Perform the Redo phase.
6. Repeat necessary operations from the log.
7. Identify incomplete transactions.
8. Perform the Undo phase.
9. Undo operations of incomplete transactions.
10. Write recovery completion records.
11. Stop.

Code :

ALGORITHM ARIES_Recovery(Log)

INPUT: Log file

OUTPUT: Recovered database

BEGIN

 // PHASE 1: ANALYSIS

 Read log from last checkpoint

 Identify:

 Active transactions

 Dirty pages

 Build Transaction Table

 Build Dirty Page Table

 // PHASE 2: REDO

 Find the earliest required log record

 FOR each log record from that point DO

 IF operation needs REDO THEN

 Repeat the operation

 END IF

 END FOR

 // PHASE 3: UNDO

 Identify transactions that were active during crash

```
FOR each incomplete transaction T DO
    Undo its operations
    Write compensation log record
END FOR
Write END records
RETURN recovered database
END
```

ARIES flow:

Crash → Analysis → Redo → Undo → Recovery Complete

Result:

Thus, the database was successfully recovered using the ARIES Analysis, Redo, and Undo phases.

8. Shadow Paging Recovery

Aim:

To implement Shadow Paging recovery using current and shadow page tables.

Algorithm:

1. Start
2. Create the Shadow Page Table.
3. Create the Current Page Table.
4. Keep the Shadow Page Table unchanged.
5. Modify copies of pages in the Current Page Table.
6. If the transaction commits, make the current table permanent.
7. If failure occurs, restore the Shadow Page Table.
8. Recover the database to its previous consistent state.
9. Stop.

Code :

ALGORITHM Shadow_Paging

```

BEGIN
    Create Shadow_Page_Table
    Create Current_Page_Table
    Shadow_Page_Table ← Current_Page_Table
    WHILE transaction is running DO
        When a page is modified:
            Create a new copy of the page
            Modify the new page
            Update Current_Page_Table
            Do NOT modify Shadow_Page_Table
    END WHILE
    IF transaction COMMITs THEN
        Make Current_Page_Table the new master table
        Delete old shadow pages
    ELSE IF system FAILURE occurs THEN
        Current_Page_Table ← Shadow_Page_Table
        Restore database
    END IF
END

```

Main idea:

The shadow page table remains unchanged. If failure occurs, use the shadow table to restore the previous consistent database.

Result:

Thus, Shadow Paging successfully recovered the database after a system failure.

9. Immediate Update Recovery

Aim:

To implement Immediate Update recovery using UNDO and REDO operations.

Algorithm:

1. Start

2. Read the log file after system crash.
3. Identify committed transactions.
4. Add committed transactions to the REDO list.
5. Identify uncommitted transactions.
6. Add uncommitted transactions to the UNDO list.
7. REDO operations of committed transactions.
8. UNDO operations of uncommitted transactions.
9. Restore the database to a consistent state.
10. Stop.

Code ;

ALGORITHM Immediate_Update_Recovery(Log)

INPUT: Log file

OUTPUT: Recovered database

BEGIN

 Read log from beginning

 FOR each log record DO

 IF transaction has COMMITTED THEN

 Add transaction to REDO list

 ELSE

 Add transaction to UNDO list

 END IF

 END FOR

 FOR each transaction in REDO list DO

 REDO all its operations

 END FOR

 FOR each transaction in UNDO list DO

 UNDO all its operations in reverse order

 END FOR

 Write recovery completion record

END

Example log:

<T1, A, old=100, new=200>

<T2, B, old=50, new=80>

<COMMIT T1>

CRASH

After crash:

- T1 → committed → **REDO**
- T2 → not committed → **UNDO**

Result:

Thus, the database was successfully recovered using UNDO for uncommitted transactions and REDO for committed transactions.

10. Database Connectivity Manager

Aim:

To design a database connectivity manager that supports connection pooling, query execution, and exception rollback.

Algorithm:

1. Start
2. Create a connection pool.
3. Get a connection from the pool.
4. Receive the SQL query.
5. Prepare and execute the query.
6. If execution is successful, commit the transaction.
7. If an exception occurs, rollback the transaction.
8. Return the connection to the pool.
9. Close the connections when the application ends.
10. Stop.

Code :

ALGORITHM Database_Connectivity_Manager

BEGIN

 Create connection pool

 WHILE application is running DO

```
TRY
    Get connection from pool
    Receive SQL query
    Prepare query
    Execute query
    IF query is successful THEN
        COMMIT transaction
        Return result
    ELSE
        ROLLBACK transaction
        Return error
    END IF
    Return connection to pool
CATCH DatabaseException
    ROLLBACK transaction
    Display error message
    Return connection to pool
END TRY
END WHILE
Close all connections
END
```

Result:

Thus, the database connectivity manager successfully handled connection pooling, query execution, commit, and exception rollback.

